

CSCE 632 ACCESSIBLE COMPUTING · MODULE 05 · LECTURE NOTES

Motor Control

Keyboard, targets, and time.

MODULE OUTCOMES

Explain the range of motor requirements.

Explain keyboard operability and focus management.

State the target size and pointer requirements.

Explain why timing requirements exclude people.

Test an interface with the mouse unplugged.

1 The range

- **Precision** — tremor, spasticity, dystonia, or an inherently unsteady pointer such as a head tracker mean **a small target is unhittable rather than merely fiddly**, which is §3's subject.
- **Range of motion** — a head pointer, a mouth stick, or a limited reach means **a target far from the current position is expensive in a way a hand on a mouse is not** (CSCE 671 Module 05's Fitts's law, where the distance term dominates).
- **The number of available inputs** — **switch access may have exactly one**, with a scanning highlight moving through the options, **so every step costs real seconds**, and **a twelve-step flow is not usable** regardless of how accessible each step is.
- **Speed and endurance** — **any timeout, any required double-click interval, any press-and-hold, and any drag is a barrier**, which is §4.
- **And whether a second simultaneous input is possible at all** — which **rules out modifier chords and multi-point gestures as sole mechanisms** (§3). **With switch access, every step has a cost** — which **turns step count from a convenience metric into an accessibility one**, and is a good reason to count the steps in your main flow.

2 Keyboard operability

REACHABLE

every interactive element receives focus
in an order that matches the visual order
with no element reachable only by mouse

VISIBLE

focus is always visible, with 3:1 contrast
and never removed without a replacement
(outline: none with nothing else is the single
commonest defect in this module)

OPERABLE

every action available from the keyboard
including drag, hover, and right-click paths

ESCAPABLE

no keyboard trap: you can always get out
modals return focus where it came from

SKIPPABLE

a skip link, or landmarks, so that repeated
navigation can be bypassed

outline: none with nothing to replace it is the single commonest defect in this module — and it makes the interface unusable while looking tidier, which is why it survives design review. A keyboard user with no visible focus indicator is navigating blind: the focus is somewhere, pressing Enter will do something, and there is no way to know what. **The five properties are reachable, visible, operable, escapable, skippable**, and they are worth memorising because they structure the whole test.

BECAUSE THE KEYBOARD IS THE SUBSTRATE EVERY OTHER INPUT SITS ON

Switch access, voice control, head pointers, eye tracking, and alternative keyboards all ultimately drive the keyboard interface — they synthesise focus moves and activations — so **keyboard operability is a precondition for every one of them** rather than one option among several.

And screen reader navigation depends on the **focus order** and on focusability, which means a **broken focus order breaks the no-vision case too** (Module 03 §2) — the two requirements are not independent.

Which makes 'unplug the mouse' the **highest-yield test in the course — it finds motor defects, screen reader defects, and switch-access defects simultaneously, and costs ten seconds to start.**

And power users want it anyway — the curb cut here is that **keyboard shortcuts and full keyboard operability are a requested feature** that people write in to ask for — **which makes this the easiest requirement in the course to fund** (Module 01 §2, and Module 12 §2's practical version).

3 Targets and pointers

- **Targets at least 24 by 24 CSS pixels**, or spaced so that a 24-pixel circle centred on each does not overlap a neighbour — and **44 is the usual platform guidance for touch, which is better** and is what to aim for rather than the floor.
- **No path-dependent gesture as the only mechanism** — a swipe, a drag, or a traced path needs a **single-pointer alternative**, because tracing a path requires sustained directional control that a tremor or a head pointer cannot provide.
- **No multi-point gesture as the only mechanism** — pinch-to-zoom needs buttons as well, since a one-finger or one-switch user cannot pinch.
- **No hover-only content** — because hover cannot be produced by touch, by keyboard, or reliably by a head pointer — and where hover content exists it must be **dismissible without moving the pointer, hoverable itself, and persistent** until dismissed.
- **And actions fired on the down-event are a trap** — **commit on up**, so that a mistaken press can be **aborted by moving away before releasing**, which is a specific criterion, is free to implement, and is what every native button already does. **Commit on the up-event, not the down-event.**

4 Time

ANY TIME LIMIT EXCLUDES ANYBODY WHO NEEDS LONGER, AND MOST TIME LIMITS ARE ARBITRARY

A session timeout, a carousel that advances on its own, a status message that disappears after three seconds, a required double-click interval, a drag-and-hold — each one assumes a speed, and the assumption is usually **unexamined** and inherited from a default.

So the rule is: **adjustable, extendable, or absent** — at minimum twenty seconds' warning and a simple way to extend — except where the limit is genuinely essential (a live auction, a real-time event) or is already very long.

And the data-loss case is the serious one: a form that times out and discards what was typed punishes slow entry specifically — which is exactly the wrong group, since slow entry is what a motor or cognitive constraint produces. Preserve the data across re-authentication.

Plus anything that moves, scrolls, blinks, or auto-updates needs a pause control — which is also **CSCE 671 Module 06's attention argument** (motion captures attention involuntarily) and **Module 06's cognitive one**, arriving at the same requirement from three directions.

AND THE TEST IS THE CHEAPEST ONE THAT EXISTS

Unplug the mouse. Then complete the three most important tasks in your interface using only the keyboard — and resist the urge to touch the trackpad, which is harder than it sounds.

Watching for the five properties — reachable, visible, operable, escapable, skippable (§2) — and recording every place you got stuck, including the places where you only got through because you knew the layout.

Then check the targets, the gestures, and the timeouts (§§3 and 4), which is a read-through of the code rather than an interactive test and takes about as long.

Which finds more real defects per minute than anything else in this course — and is the first thing Project 1 asks for, because it reframes how you think about every control you build afterwards, which is a larger effect than the defect list itself.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
The WAI keyboard and forms tutorials (free)	§2 — focus order, focus visibility, and the markup that produces them without effort.
Understanding WCAG 2.5.x on pointer input (free)	§3's criteria — target size, pointer gestures, pointer cancellation, and dragging movements, each with its reasoning.
Understanding WCAG 2.2.1 on timing adjustable (free)	§4 — including the exceptions, which are narrower than people assume.
The Inclusive Components patterns (free)	§§2 and 3 as working components — keyboard models and focus management, built up from the mechanism.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Name the five varying factors** and one requirement each implies.
- 2 **Count the steps** in your interface's main flow, and halve them.
- 3 **Unplug the mouse** and complete three tasks.
- 4 **Check all five keyboard properties** and report which failed.
- 5 **Grep your stylesheets for outline** and audit every suppression.

- 6 **Open and close a modal by keyboard** and check where focus goes.
- 7 **Measure every target** in your interface against 24 and 44 pixels.
- 8 **Find a hover-only, drag-only, or pinch-only mechanism** and add an alternative.
- 9 **List every time limit** in something you built, and justify or remove each.
- 10 **Check whether a timeout discards typed data**, and fix it if so.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five varying motor factors and their implications.
- 2 Why does step count matter for switch access?
- 3 Give the five keyboard properties.
- 4 What is the commonest defect, and why does it survive review?
- 5 Why is the keyboard the substrate for other inputs?
- 6 Why is unplugging the mouse the highest-yield test?
- 7 State the target size requirement, both forms.
- 8 Name three gesture requirements.
- 9 Why commit on the up-event?
- 10 State the timing rule, its exceptions, and the data-loss case.